A Rights-Safe Local Soccer Analytics Pipeline: From Licensed Footage to Match-Outcome Prediction

The Soccer-Edge Collaboration
`github.com/scottjoyner/soccer-analytics`

Draft — July 2026

Abstract

We present an open, reproducible pipeline for soccer analytics that enforces data-rights compliance by construction while turning permitted *local* footage and open event data into structured tables, detection datasets, and match-outcome models. The system ingests open event feeds (StatsBomb Open Data, Metrica Sports sample data, and approved SoccerNet subsets) together with rights-approved local video, runs a YOLO-based computer-vision abstraction to detect players and the ball, aggregates per-player and per-team statistics, and fine-tunes match-outcome predictors (winner classification and home/away score regression) as well as a convolutional occupancy-grid model. A lineage-and-rights gate refuses to process any footage that lacks a recorded written-rights reference, so no scraped or unlicensed audiovisual content can enter the pipeline. We report a reproducible case study over 98 permitted FIFA World Cup 2026 highlight clips, yielding 88,127 detection rows, and we discuss the honest limitation that short highlight-clip detection aggregates carry little predictive signal for match outcome.

1 Introduction

Data-driven soccer analytics has advanced rapidly, but most pipelines assume unfettered access to broadcast footage and event feeds. For academic research this assumption is increasingly untenable: audiovisual rights are tightly held, and scraping or caching copyrighted video violates both platform terms and the data-governance expectations of institutional review. We argue that a *rights-safe, local-first* design is not merely a compliance footnote but a first-class system property that improves reproducibility: if the pipeline can only consume data the researcher is entitled to process, then every published result is, by construction, re-runnable by others with equivalent rights.

This paper describes Soccer-Edge, a pipeline organized around three invariants: (i) public video URLs are *discovery metadata only* and are never processing inputs; (ii) every processed table and derived feature table carries lineage fields and an explicit `rights_reference`; and (iii) a single rights gate is enforced at every footage entry point—including a local screen/webcam/image *capture* intake that is itself rights-gated. We validate the system end-to-end on 98 permitted local highlight clips and open event data, and we describe an evaluation-to-promotion gate that refuses to promote a model unless it demonstrates measured out-of-sample lift.

2 Related Work

Event-stream datasets such as StatsBomb Open Data [StatsBomb], Metrica Sports sample tracking [Metrica], and SoccerNet [SoccerNet] have enabled reproducible spatiotemporal modeling without audiovisual rights. Prior computer-vision work detects players, the ball, and team possession from broadcast video using object detectors such as YOLO [Redmon, Ultralytics] and tracks them across frames. Our contribution is not a new detector but a *governance wrapper*: a reproducible, rights-gated pipeline that composes existing open components (Ultralytics YOLO, scikit-learn, PyTorch) into a research workflow that provably cannot ingest unlicensed media.

3 System Design and the Rights Model

The pipeline models provenance explicitly. Every processed table records `lineage_source_name`, `lineage_source_path`, `lineage_dataset_version`, and `lineage_ingested_at_utc`. Footage rows carry a `rights_status` drawn from {`owned`, `licensed`, `compatible_license`, `pending`, `blocked`}, and any approved status *requires* a non-empty `rights_reference` (an explicit written-rights pointer such as a permit file or a recorded consent).

A single assertion, `assert_processable`, is reused across the command surface: `discover video` (metadata only), `catalog-local`, `video plan`, `detect-yolo`, `process`, `process-local-model`, `export-frames`, `train player-ball`, `train local-finetune`, and the `capture` intake commands. The rights gate is *mandatory* for every footage-processing command: it requires both `--manifest` and `--video-id`, and the footage file is opened only if its `rights_reference` is recorded, its `local_path` *exists*, *and* that path matches the input. The underlying `run_yolo_detection` call enforces the same gate internally, so un-gated footage cannot be processed via the library either. This makes the rights check a hard, test-covered gate rather than documentation.

We further harden the gate with a *modality blocklist* (`configs/modality_rules.json`) enforced inside `validate_processable_video`: any manifest row whose `source_url` or `clip_type` references a blocked modality (`youtube`, `youtu.be`, `twitch`, `stream`, or any `http(s)/rtmp/rtsp` scheme) is rejected outright. This guarantees that public or remote URLs stay discovery metadata only and can never become processing inputs, even if a manifest row is otherwise well formed.

Rights-gated capture intake. To let a researcher process footage they are entitled to, the pipeline provides a local *capture* intake (`capture screen | webcam | image`) that records the screen, a webcam, or a single screenshot to a licensed captures directory and appends a rights-gated manifest row. Every capture requires an approved `rights_status` and a recorded `rights_reference`, and the captured source is written with a local `capture://` scheme so the same modality blocklist applies—capturing a third-party stream remains prohibited. A real-time mode (`--detect`) runs the YOLO detector on each frame as it is recorded, emitting a per-frame detections table (and, optionally, an annotated video) so footage is processed *as* it is captured rather than only after saving; the resulting file and detections feed the same rights-gated fine-tuning path. A single command (`capture to-match-predictor`) wires an approved clip end-to-end into the match-outcome model: rights-gated YOLO detection, per-match feature aggregation, and a `match_id`-keyed merge with open-event features (xG/xT/pressure) from StatsBomb, Metrica, OpenFootball, and football-data, so captured footage trains alongside the other data sources without leaking frames across the train/validation split. The merged event features are passed through to the trainer (not silently dropped), and the footage input gate refuses remote/streaming URLs and resolves symlinks before any frame is opened, so a symlink inside the licensed root cannot smuggle in an unapproved file.

4 Data

Open event data. StatsBomb Open Data, Metrica sample events/tracking, approved SoccerNet subsets, OpenFootball match-result archives, and football-data.co.uk historical results are ingested into normalized Parquet tables with lineage. From these we derive per-match player statistics (shots, passes, carries, pressures, interceptions, tackles, fouls) and cross-match aggregates with per-player totals, per-match averages, and team/opponent splits.

Permitted local footage. For this study we use 98 FIFA World Cup 2026 highlight clips that were provided to the authors as permitted local files. Their match results (teams, scores, penalty shootouts) are encoded directly in the filenames and were parsed into a structured results table; no audiovisual content was scraped or downloaded.

Codec handling. The provided clips are AV1-encoded. The local media reader has no AV1 decoder, so each clip is first transcoded to H.264 with FFmpeg and then passed to the detector. This transcode step is a rights-preserving local transformation of already-permitted footage; the original files remain the system of record and the `rights_reference` propagates to every derived table.

5 Computer-Vision Abstraction

Detections are produced by a `YOLODetector` wrapping Ultralytics YOLOv8n [Ultralytics], exposed through the `detect-yolo` command. For each clip it emits per-frame player/ball bounding boxes, frame-to-frame tracks, and pitch-space ball/player states. From these we compute, per match, aggregate features

$$f = (\texttt{n\_player},\ \texttt{n\_ball},\ \texttt{avg\_det\_per\_frame},\ \texttt{ball\_center\_x},\ \texttt{ball\_center\_y})$$

normalized to the frame. We additionally rasterize each frame into an occupancy grid (channels $\times$ height $\times$ width, default $3 \times 8 \times 8$) for the convolutional model.

6 Feature Aggregation

Player statistics are aggregated with `build_player_aggregates`, which yields cross-match totals, per-match averages, appearances, and an overall pass completion rate, with optional `split_by=team` and `split_by=opponent` breakdowns. Rate and count features are additionally normalized to a *per-90-minutes* basis via `normalize_per_90` so that players with different minutes are comparable, and `expected_starter_flag` encodes lineup-aware expected-starter status (defaulting to an unknown state when no lineup is supplied). Player features are rolled up to the team level with `aggregate_roster_to_team`, which adds team-mean/min/max aggregates and an expected-starter count. For the highlight study, filename parsing produced a 98-match results table (50 distinct teams) that supplies the supervisory labels (winner, home/away score) for the outcome models.

7 Match-Outcome Models

Tabular model. `train_match_predictor` fits a multinomial logistic-regression winner classifier and two RandomForest regressors for the home and away scores, using the five detection-derived features above. To avoid optimistic in-sample estimates we evaluate behind a leakage-safe stratified train/test split (68/30), a `StandardScaler`, and a `CalibratedClassifierCV` wrapper so the winner probabilities are calibrated and the Brier score is meaningful.

Convolutional model. `train_match_predictor_cnn` builds an NPZ of occupancy-grid tensors (grouped by match, with a sliding window of sequence length 4) and trains a small CNN winner classifier (PyTorch). The same grid tensors feed an out-of-sample evaluation (`scripts/evaluate_cnn.py`) that splits matches into a stratified 68/30 train/test hold-out and trains the CNN only on train, reporting held-out sequence- and match-level accuracy.

Evaluation-to-promotion gate. Model promotion is treated as a governed, test-covered decision rather than a manual step. An `eval-to-metrics` bridge normalizes each evaluation script's `metrics.json` into a flat predictive-metrics table (accuracy, Brier), and a promotion gate (`model promotion-gate`) checks card validity, dataset versioning, annotation audits, and object metrics *and* enforces two predictive criteria: the candidate must beat the majority-class baseline by a configurable accuracy margin and stay under a maximum Brier threshold. Only a passing bundle can be promoted (`model promote`) into a versioned promoted-model directory with its cards, metrics, and gate report; a failing bundle exits non-zero and writes nothing. Consequently the highlight-clip models below, which show no out-of-sample lift, are structurally blocked from promotion until a rights-clean source supplies genuine lift.

8 Case Study and Results

Over the 98 permitted clips the batch produced **88,127** detection rows (86,007 player detections, 1,541 ball detections) across 7,878 grid frame-rows. The tabular model was trained on the resulting 98-match dataset.

We compare two feature sets: (i) the five *count* features used above (`n_player`, `n_ball`, `avg_det_per_frame`, `ball_center_x/y`), and (ii) nine *track* features that attempt to capture possession and pressure from the per-frame boxes—ball frame-rate, person frame-rate, mean nearest player–ball distance, mean players-near-ball, contested-ball rate, ball movement, and mean player-box area. Both are evaluated behind the same stratified 68/30 split with a calibrated winner classifier.

Table 1: Out-of-sample results on the 98-match highlight dataset (30-match held-out test, calibrated classifier). Accuracy is reported against the majority-class baseline (49%); a Brier score near 0.5–0.7 indicates near-random probability quality.

Feature set / metric	Count	Track	Note
Winner accuracy (test)	53.3%	50.0%	baseline 49.0%
Winner Brier (test)	0.623	0.614	lower is better
Home-score MSE (test)	1.770	1.864	out-of-sample
Away-score MSE (test)	2.345	2.066	out-of-sample
Train rows / test rows	68 / 30		stratified
Detection rows	88,127		86,007 player / 1,541 ball
Grid frame-rows	7,878		CNN input

Open event-data model. To show where predictive signal actually lives, we applied the same calibrated evaluation pipeline to StatsBomb Open Data (Premier League 2023/24; 380 matches, sanctioned tier-1 open event data, non-audiovisual). Features are derived home/away from the event stream: expected goals (xG, provided), a data-driven expected-threat (xT) surface fit from the possession-transition matrix, shots, pressure regains, and count features (passes, pressures, carries, clearances, interceptions, blocks, fouls, ball recoveries). Under repeated 5-fold $\times$ 10 cross-validation the calibrated winner classifier reaches **63.3% (±2.9)** accuracy with a **0.164** Brier and home/away score MSE of **0.96 / 0.87**, against a **41.3%** majority-class baseline. Adding xT and pressure regains (63.1% ±3.6) does not materially change accuracy, indicating the basic xG-plus-count feature set already captures the available signal. This confirms the pipeline yields genuine, calibrated lift once fed rights-clean open supervision—precisely the signal source the highlight-clip features lack.

Honest limitation. As Table 1 shows, neither the count nor the track detection features outperform the majority-class baseline for winner prediction, and the calibrated Brier scores (≈ 0.61) confirm the probabilities are no better than chance. The out-of-sample CNN evaluation agrees: over a 5-fold repeated-CV sweep (10 epochs) it reaches 49.3% (±1.8) sequence accuracy and 48.9% (±1.3) match accuracy—both at the majority-class baseline—with a winner Brier of 0.630 (±0.011). These figures are produced by the repeated-CV variant of the evaluation (`--folds`/`--repeats`), which reports mean±sd across a stratified match hold-out; the deep model therefore reproduces the same null result as the tabular features, rather than uncovering signal the counts miss. This is the expected and, we argue, informative result: short highlight reels are highlight-selected (not representative of open play), contain far fewer frames than a full match, and aggregate detection counts are nearly invariant to final score. The pipeline therefore demonstrates end-to-end reproducibility and governance—including a leakage-safe, calibrated evaluation—while clearly delineating where predictive signal must come from: full-match footage or rich open event data (Section 4). The open event-data result above shows the same method succeeds once appropriate supervision is supplied. We report out-of-sample figures only and do not claim generalization.

9 Reproducibility

The study is scripted and re-runnable: `scripts/process_highlights_batch.py` (transcode + detect, resumable), `scripts/build_highlights_training.py` (aggregate + fine-tune), `scripts/evaluate_highlig` (out-of-sample tabular + calibrated evaluation), `scripts/evaluate_cnn.py` (out-of-sample CNN winner evaluation, stratified 68/30 match hold-out), and `scripts/batch_cnn_eval.sh` (seed-sweep wrapper for off-box batch compute), plus a repeated-CV variant `scripts/evaluate_cnn.py --folds 5 --repeats 1` that reports mean±sd across folds. A data card and a zipped bundle of the processed tables ship with the repository, and every derived table carries lineage and `rights_reference` fields. The pipeline's graph-export payloads (Neo4j JSONL) now cover dataset versions, annotation audits, object-evaluation summaries, and *player-match* and *player-form* feature nodes, so derived features are themselves lineage-tracked; the fine-tuning and evaluation stack runs CPU-only on the batch machine (PyTorch CPU, capped threads).

10 Limitations and Ethics

The system intentionally cannot ingest scraped or unlicensed media; researchers remain responsible for confirming their own rights to any local footage. The modeling results are illustrative and based on a small, highlight-only corpus; they are not a predictive claim about match outcomes. No external execution or real-money actions are performed.

11 Conclusion and Future Work

We presented a rights-safe, local-first soccer analytics pipeline that turns permitted footage and open event data into detection datasets and match-outcome models, with a hard, tested rights gate at every footage boundary—extended to a rights-gated screen/webcam/image capture intake with an optional real-time detection mode. An evaluation-to-promotion gate closes the loop by admitting only models with measured out-of-sample lift into a versioned promoted-model store. Future work includes calibrating the detector on full-match footage. Per-fold xT surface fitting (eliminating even the negligible league-wide leakage), tracking-derived possession features, the feature scaler, calibrated classifier, repeated cross-validated evaluation, per-90/lineup-aware player features, the modality-blocklist rights gate, rights-gated capture intake, the evaluation-to-promotion gate, an open event-data model with expected threat, and player-match/player-form graph payloads are already in place.

References

[StatsBomb] StatsBomb. *StatsBomb Open Data.* `https://github.com/statsbomb/open-data`.

[Metrica] Metrica Sports. *Sample Game Data.* `https://github.com/metrica-sports/sample-data`.

[SoccerNet] A. Cioppa et al. *SoccerNet: A Scalable Dataset for Action Spotting in Soccer Videos.* CVPR 2020.

[Redmon] J. Redmon, S. Divvala, R. Girshick, A. Farhadi. *You Only Look Once: Unified, Real-Time Object Detection.* CVPR 2016.

[Ultralytics] Ultralytics. *YOLOv8.* `https://github.com/ultralytics/ultralytics`.

[scikit] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python.* JMLR 2011.

[PyTorch] A. Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library.* NeurIPS 2019.